

DATA 622: Lab 1: Bias Variance Trade-Off

Instructions:

Complete this assignment by answering the following questions using code, text descriptions, and mathematics in a quarto markdown document. Render your .qmd to a pdf and submit both the qmd and pdf files on brightspace.

The purpose of this assignment is for you to review basic data manipulation and plotting in python, make sure that you have working software tools to complete the assignment, and learn a little bit about using sci-kit learn to explore the bias-variance tradeoff.

Problem 1: Exploring the Auto data set

This exercise involves the `Auto` data set studied in the interactive lab in chapter 2 of ISLP. Make sure that the missing values have been removed from the data.

- (a) Which of the predictors are quantitative, and which are qualitative?
- (b) What is the range of each quantitative predictor? You can answer this using the `min()` and `max()` methods in `numpy`.
- (c) What is the mean and standard deviation of each quantitative predictor?
- (d) Now remove the 10th through 85th observations. What is the range, mean, and standard deviation of each predictor in the subset of the data that remains?
- (e) Using the full data set, investigate the predictors graphically, using scatterplots or other tools of your choice. Create some plots highlighting the relationships among the predictors. Comment on your findings.
- (f) Suppose that we wish to predict gas mileage (`mpg`) on the basis of the other variables. Do your plots suggest that any of the other variables might be useful in predicting `mpg`? Justify your answer.

Problem 2: kNN Classification of the ZIP code digit data

In order to complete this problem you will need to download `zip.train` and `zip.test` from the course github page. These datasets contain images of hand-drawn digits. We will be experimenting with kNN classification and factors impacting the bias-variance trade-off, and this will also be chance to practice using `scikit-learn`.

- (a) Download and load the training and test data sets using pandas. Make sure to load all of the data (there is no header) The zeroth column corresponds to the class label, a digit from 0-9, and the columns 1 to 256 correspond to a grayscale value from -1 to 1. Select the first entry in the training set, resize it to 16x16, and plot the image (you can use `plt.imshow()`).
- (b) The following code imports the kNN classification function from `scikit-learn` as well as an accuracy function, trains a kNN classifier, and tests its accuracy on hypothetical training and testing data:

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

knn = KNeighborsClassifier(n_neighbors=4)
knn.fit(zip_image_train, zip_class_train)

zip_pred_train = knn.predict(zip_image_train)
accuracy = accuracy_score(zip_class_train, zip_pred_train)
print(f'Accuracy: {accuracy:.4f}')
```



```
zip_pred_test = knn.predict(zip_image_test)
accuracy = accuracy_score(zip_class_test, zip_pred_test)
print(f'Accuracy: {accuracy:.4f}')
```

Where the dataframes contain just the training and testing images and class.

Adapt this code to determine if we can observe the bias variance trade-off for different numbers of neighbors k . Specifically, recreate plot 2.17 (pay attention to the x-axis scale and use the same choice as in the book) which shows test and train classification accuracy as a function of $1/k$. Select a range of k from 1 to 300. You do not have to plot every single k value in this range if the problem is computationally intensive on your machine. Do you observe a U -shaped curve in the testing error (and a divergence from training error) as $1/k$ increases?

- (c) Introduce some noise in the training and testing labels for both the training and testing data. You can do this by using `np.random.choice` to sample from the range of indices of each of the training and test set to determine which labels will be changed, and `np.random.choice` again to pick the new label. After making this modification, repeat problem (b). How did adding label noise impact the shape of the testing and training error versus $1/k$ curves?